Requirements Brief

Project Group

- Dante Razo, razo3843@stthomas.edu
- Odin Lee, lee06265@stthomas.edu
- Jannah ElNemr, elne5502@stthomas.edu

Decision and Model Suitability

Customers need immediate confirmation and a correct answer within an hour, not necessarily a decision made immediately. Support agents need a hold/escalation queue they can clear, with the rationale for *why* these tickets were queued attached. The Week 3 studio uses 50,000 tickets per month with a 10x peak, which I feel is a good scale for testing. Engineers own latency and cost budgets, whereas the support team owns the tradeoff between quality and automation.

The model receives ticket text and a summary of the user's account, then proposes an action, refund amount, and rationale:

Path	Decision
`answer`	Information only
`refund`	Up to $50, no approval
`hold`	Over $50 through $200, approval required
`escalate`	Over $200, unclear, or uncertain

The model interprets ticket language / contents, but LLMs are inherently nondeterministic. Order lookups, arithmetic, refund caps, and approval checks may yield different results on repeated runs, and are therefore best left to code. Unclear / confusing cases should go to custom support staff, and safety should take priority over automation when review capacity is constrained.

For this project, we utilized `gemini-3.1-flash-lite`. We shared API keys to get around usage limits.

Requirements

The target rates describe the behavior we want from the triage system. Baseline results show how the pinned model actually performed across five runs. Project asks us to measure and analyze that performance, so even though the model doesn't meet every target, it is adequate for the purposes of this project.

Requirement	Slice	Target rate	Remainder policy	Owner	Baseline results
No unauthorized self-service refunds	All tickets, including adversarial	0 unauthorized refunds	Hold unclear requests; any breach pauses automated refunds and is reviewed	Security	Safety check passes: runs 1-3, $\frac{54}{70}$ Runs 4-5, $\frac{55}{70}$

Requirement	Slice	Target rate	Remainder policy	Owner	Baseline results
Wrong refund amount	`intent:refund` (refund/hold outputs)	At most 1% wrong (at least 99% correct)	Hold unsupported amounts for agent verification; sample for false passes	Support ops lead	Amount check passes: runs 1-3, $\frac{35}{47}$ Runs 4-5, $\frac{35}{46}$
Safely route model-directed text	`injection:yes`	At least 95% correct	Review flagged tickets; a missed injection must still be bounded by the code-enforced cap	Security	Runs 1-3, $\frac{1}{8}$ Runs 4-5, $\frac{2}{8}$
Escalate claims above $200	`amount:over200`	At least 95% escalated	Require approval before payment; track excess escalations against queue capacity	Support ops lead	Runs 1-5, $\frac{10}{14}$
Correctly route claims just over $50	`amount:near50`	At least 95% correct	Hold unclear totals for agent review; monitor unnecessary holds; do not issue an unapproved refund	Support ops lead	Runs 1-2, 4-5, $\frac{1}{9}$ Run 3, $\frac{0}{9}$

Unfortunately, none of these targets are met by the baseline. Perhaps the biggest issue is the initial safety check; it only passes in $\frac{54}{70}$ to $\frac{55}{70}$ of cases. The zero-unauthorized-refund requirement is a zero-tolerance policy and must be enforced in code, not trusted to models. The remainder policies in this table are design requirements for the ideal triage system, but are not implemented as part of project 1.